Object Oriented Programming through cpp

Unit-3

1 Distinguish between overloading using a member function and a friend function.

2 Explain binary operator overloading with a suitable example.

3 Explain operator overloading using friend functions with syntax and an example.

4 Explain different types of inheritance supported by C++ with examples.

5 Explain the use of Virtual Functions with an example program.

6 What is an abstract class? Demonstrate with an example.

1.Distinguish between overloading using a member function and a friend function.

Operator overloading allows an operator such as +, -, ==, etc., to work with user-defined objects.

Member Function	Friend Function
1. The operator function is defined inside the class.	1. The operator function is declared as a friend inside the class but defined outside.
2. It is called using the left-hand object.	2. It can be called with objects passed as arguments.
3. It accesses private/protected members directly.	3. It can access private/protected members because it is declared as a friend.
4. It generally requires one less parameter for binary operators.	4. It generally requires two parameters for binary operators.
5. *this represents the left operand.	5. Both operands are explicitly passed as arguments.

Example:

```
#include <iostream>
using namespace std;
class Demo {
   int x;
public:
```

```
    Demo(int a) { x = a; }
    // Member function
    Demo operator+(Demo d) {
        return Demo(x + d.x);
    }
    // Friend function
    friend Demo operator-(Demo a, Demo b);
    void show() { cout << x; }
};
Demo operator-(Demo a, Demo b) {
    return Demo(a.x - b.x);
}
int main() {
    Demo d1(20), d2(10);
    Demo d3 = d1 + d2;  // Member
    Demo d4 = d1 - d2;  // Friend
    d3.show();
    cout << endl;
    d4.show();
}
```

Output: 30

2.Explain binary operator overloading with a suitable example.

Binary operator overloading is the process of giving a special meaning to a binary operator such as +, -, *, /, etc., when it is used with objects of a class.

A binary operator works on two operands. In C++, it can be overloaded using a member function or friend function.

Example:

```
#include <iostream>
using namespace std;
class Demo {
   int x;
public:
   Demo(int a) {
      x = a;
   }
   // Binary + operator overloading
   Demo operator+(Demo d) {
      return Demo(x + d.x);
   }
   void display() {
      cout << "Result = " << x;
   }
};
int main() {
   Demo d1(10), d2(20);
```

```
    Demo d3 = d1 + d2;
    d3.display();
    return 0;
}
```

Output:30

3.Explain operator overloading using friend functions with syntax and an example.

Operator overloading using a friend function means defining an overloaded operator as a friend function of a class. A friend function is not a member of the class, but it can access the class's private and protected data members.

Syntax:

```
class ClassName {
    // data members
    friend ReturnType operator op(ClassName, ClassName);
};
```

Example:

```
#include <iostream>
using namespace std;
class Demo {
    int x;
public:
```

```
    Demo(int a) {
        x = a;
    }
    friend Demo operator+(Demo d1, Demo d2);

    void display() {
        cout << "Result = " << x;
    }
};
// Friend function
Demo operator+(Demo d1, Demo d2) {
    return Demo(d1.x + d2.x);
}
int main() {
    Demo d1(10), d2(20);
    Demo d3 = d1 + d2;
    d3.display();
    return 0;
}
```

---

4 Explain different types of inheritance supported by C++ with examples.

Inheritance is an OOP concept in which a new class (derived class) acquires the properties and functions of an existing class (base class). It promotes code reusability.

C++ supports the following types of inheritance:

1. Single Inheritance

One derived class inherits from one base class.

```
class A {
public:
  void show() {
    cout << "Base class";
  }
};
class B : public A {
};
```

2. Multiple Inheritance

One derived class inherits from more than one base class.

```
class A { };
class B { };
class C : public A, public B {
};
```

3. Multilevel Inheritance

A class is derived from another derived class, forming a chain.

```
class A { };
class B : public A { };
class C : public B { };
```

4. Hierarchical Inheritance

Multiple derived classes inherit from one base class.

```
class A { };
```

class B : public A { };

class C : public A { };

5. Hybrid Inheritance

It is a combination of two or more types of inheritance, such as multiple and multilevel inheritance.

class A { };

class B : public A { };

class C : public A { };

class D : public B, public C { };

---

5. Explain the Use of Virtual Functions with an Example Program.

A virtual function is a member function in a base class that is declared using the keyword virtual. It allows function overriding and supports runtime polymorphism.

Uses of Virtual Functions:

- They allow the derived class function to be called using a base class pointer.
- They provide runtime polymorphism.
- They allow different derived classes to provide their own implementation of the same function.

Example Program:

```
#include <iostream>
```

```
using namespace std;
class Base {
public:
   virtual void show() {
      cout << "Base class";
   }
};
class Derived : public Base {
public:
   void show() {
      cout << "Derived class";
   }
};
int main() {
   Base *ptr;
   Derived obj;
   ptr = &obj;
   ptr->show();
   return 0;
}
```

Output:

```
Derived class
```

---

6. What is an Abstract Class? Demonstrate with an Example.

An abstract class is a class that contains at least one pure virtual function. It is used as a base class and cannot be used to create objects directly.

A pure virtual function is declared by assigning = 0.

Syntax:

```
class ClassName {
public:
   virtual void functionName() = 0;
};
```

Example Program:

```
#include <iostream>
using namespace std;
class Shape {
public:
   virtual void draw() = 0;   // Pure virtual function
};
class Circle : public Shape {
public:
   void draw() {
      cout << "Drawing Circle";
   }
};
int main() {
   Circle c;
   c.draw();
```

```
    return 0;
}
```

Output:

```
Drawing Circle
```